



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

RESEARCH TRACK II

Assignment RT2

Statistical analysis

Authors:

BEAUJEAN Bertille

Student ID:

s7899816

Professors:

Carmine Tommaso Recchiuto

May 12, 2025

Contents

1	Introduction	2
1.1	Original environment	2
1.2	Previous improvements	2
1.3	Analysis of an alternative implementation	2
2	Hypotheses	3
2.1	Criteria of comparison	3
2.2	Basic examples	3
2.3	Null and alternative hypothesis	4
3	Experimental setup	4
4	Results	4
5	Statistical analysis	4
6	Conclusion	5

1 Introduction

This assignment is the continuation of the environment developed in Research Track I and of the previous assignments of Research Track II.

1.1 Original environment

The assignment of Research Track I aimed at developing a ROS environment (*noetic*) in which a robot moves in a *gazebo* simulation with obstacles.

As a result of this assignment, we had:

- a window with gazebo simulating the robot and its environment ;
- a window with Rviz ;
- a terminal showing the position (x,y) and the velocity (Vx,Wz) of the robot every second pub_pos_vit.py (pub/sub using a custom message built from the topic /odom);
- a terminal to set or cancel new targets, with a message explaining how to do so set_target.py (action client using feedback status to know when the target has been reached);
- a terminal for the output of the service node last_target, which shows the coordinates (x,y) of the last target sent by the user (service returning the coordinates of the last target when called). The node should be called on a terminal by **rosservice call /last_target** whenever the user wants information on the last node.

1.2 Previous improvements

In the previous assignments of Research Track II, the terminal used to set or cancel targets was replaced by an interface in a *jupyter notebook*.

This notebook, enables to see the current position of the robot and its distance to the closest obstacle. It gives an interface with buttons and other widgets to set and cancel targets. Then, the trajectory of the robot in the environment is dynamically plotted in a graph and the number of targets reached or not reached is displayed.

1.3 Analysis of an alternative implementation

In the precedent environment, the robot was reaching targets while avoiding obstacles with a "bug" algorithm.

It was arbitrarily chosen that the robot would move around the obstacles by going to the left.

Now, we want to try to make the robot move around the obstacles by going to the right, and compare the results. Let us perform a statistic analysis to compare the two different implementations.

2 Hypotheses

2.1 Criteria of comparison

The goal is to assess the efficiency of a path planning algorithm which sends a robot from a point to another. We want to find the fastest and shortest way to the target point. In other words, the efficiency of the algorithm is assessed by the time and distance that the robot takes to reach the target. The best algorithm will be the one which can provide the fastest and shortest path for given starting point and target point. In the real world, the robot would spend energy when moving forward, but also when turning on itself, so the distance traveled by the robot might not be the best indicator.

Thus, the best criterion of efficiency for this comparison seems to be the time needed by the robot to go from a given starting point to a given target point.

In the following parts, we will call the "best" of the two alternatives the fastest path, or equally the shortest path.

2.2 Basic examples

A priori, for some specific obstacles, it might be more efficient to go around it by turning to the left or to the right.

For example, in the case of figure 1, we can easily state that, to go from the green dot to the red cross, the shortest distance would imply to move around the obstacle by heading left, according to the green line.

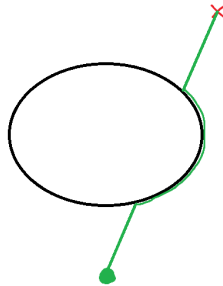


Figure 1: Situation where turning left or right makes a difference

However, if we consider the inverse problem, if the robot had to go from the red cross to the green dot, the robot had better turn in the opposite way to take the shortest path, that is to say the green one again.

Also, if the starting point and the target point are located in a symmetry plan of the obstacle, this time, we can easily state that turning left or right will not modify the distance travelled by the robot. We can see in figure 2 that the paths green and blue are equivalent in distance.

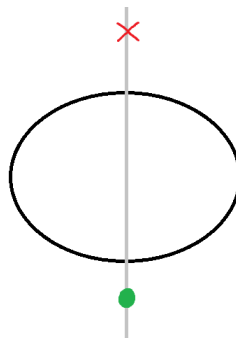


Figure 2: Situation where both alternatives are equivalent

Indeed, it seems that, in general, for different starting points, different target points and different looking obstacles, the best option would be alternatively to turn left or right.

Thereby, I think that, with a sufficient number of experiments, both alternatives would be equivalent.

2.3 Null and alternative hypothesis

As we are to compare the new method (turning right) with the previous one (turning left) about its superiority, we will define:

- The null hypothesis: The two methods don't provide equivalent time to reach a target.
- The alternative hypothesis: The two methods provide equivalent time to reach a target.

As the power of my computer might not enable to perform a high number of experiments, it might be better to compensate by allowing a level of significance lower than usual, for example, equal to 15 or 20%.

We will try to accept the alternative hypothesis by rejecting the null hypothesis.

3 Experimental setup

We want to measure the time taken by the robot to go from a point to another, with each method. Thus, we should perform a Paired T-Test with the "Turning Left" method and the "Turning Right" Method.

To do so, a list of successive targets will be set in the limits of the environment. For each method, the targets will be sent one by one, in the same order, until every target is reached. We will measure the time needed to go from a point to another.

For each section of the trajectory, we will save the time needed by both methods, to perform the statistical analysis.

The modified code is provided in the GitHub repository, in Notebooks, under the name of "Statistic analysis". The code to perform the T-Test is written in same folder, and the data gotten from the experiments is also in the same folder, with explicit names.

4 Results

The experiment is done with 5 random targets, generated according to a normal law (*random.randint*).

The robot is initially located in (0,1).

x	y	duration (left)	duration (right)	difference of duration
-4	-7	73.62	64.51	9.11
7	3	156.36	47.90	108.45
5	-9	34.15	36.58	-2.43
-6	1	89.42	101.34	-11.93
0	0	23.77	25.55	-1.79

Table 1: Duration (seconds) to reach targets on a random set of targets

5 Statistical analysis

To realize the paired t-test, we need to compute the t-statistic.

We can compute the following statistic data:

- Number of experiments: nb_exp = 5
- Degree of freedom: DoF = 4
- Mean difference: d_mean = 20.28210482597351
- Standard deviation: sd = 44.59
- Standard error: SE_d = 19.94
- t-statistic: T = 1.017

		75%	80%	85%	90%	95%	97.5%	99%	99.5%	99.75%	99.9%	99.95%	
	One Sided												confidence level
	Two Sided	50%	60%	70%	80%	90%	95%	98%	99%	99.5%	99.8%	99.9%	
1		1.000	1.376	1.963	3.078	6.314	12.71	31.82	63.66	127.3	318.3	636.6	
2		0.816	1.061	1.386	1.886	2.920	4.303	6.965	9.925	14.09	22.33	31.60	
3		0.765	0.978	1.250	1.638	2.353	3.182	4.541	5.841	7.453	10.21	12.92	
4		0.741	0.941	1.190	1.533	2.132	2.776	3.747	4.604	5.598	7.173	8.610	
5		0.727	0.920	1.156	1.476	2.015	2.571	3.365	4.032	4.773	5.893	6.869	
6		0.718	0.906	1.134	1.440	1.943	2.447	3.143	3.707	4.317	5.208	5.959	
7		0.711	0.896	1.119	1.415	1.895	2.365	2.998	3.499	4.029	4.785	5.406	
8		0.706	0.889	1.108	1.397	1.860	2.306	2.896	3.355	3.833	4.501	5.041	
9		0.703	0.883	1.100	1.383	1.833	2.262	2.821	3.250	3.690	4.297	4.781	
10		0.700	0.879	1.093	1.372	1.812	2.228	2.764	3.169	3.581	4.144	4.587	
11		0.697	0.876	1.088	1.363	1.796	2.201	2.718	3.106	3.497	4.025	4.437	
12		0.695	0.873	1.083	1.356	1.782	2.179	2.681	3.055	3.428	3.930	4.318	
13		0.694	0.870	1.079	1.350	1.771	2.160	2.650	3.012	3.372	3.852	4.221	
14		0.692	0.868	1.076	1.345	1.761	2.145	2.624	2.977	3.326	3.787	4.140	
15		0.691	0.866	1.074	1.341	1.753	2.131	2.602	2.947	3.286	3.733	4.073	
16		0.690	0.865	1.071	1.337	1.746	2.120	2.593	2.921	3.252	3.686	4.015	
17		0.689	0.863	1.069	1.333	1.740	2.110	2.567	2.898	3.222	3.646	3.965	
18		0.688	0.862	1.067	1.330	1.734	2.101	2.552	2.878	3.197	3.610	3.922	
19		0.688	0.861	1.066	1.328	1.729	2.093	2.539	2.861	3.174	3.579	3.883	
20		0.687	0.860	1.064	1.325	1.725	2.086	2.528	2.845	3.153	3.552	3.850	

Figure 3: Student table

Thereby, if we analyze this data with the Student table of the lectures (see figure 3), we can compare the t-statistic with the value of the table for 4 degrees of freedom, in a two-tailed test, because we are considering the difference between the two durations, should it be positive or negative.

Indeed, we have $T = 1.017 < 1.190$, so the null hypothesis is rejected with a confidence level of 70%.

This means that we cannot say that the two methods provide significantly different durations to reach a target.

By refusing the null hypothesis, we accept the alternative hypothesis.

6 Conclusion

In conclusion, the two-tailed paired T-Test done on a set of 5 individual experiments enabled to reject the null hypothesis, and thus, to accept the alternative hypothesis - that the robot can avoid obstacles by moving around it by the left or by the right in a similar time - with a confidence level of 70%.

To accept a better confidence level, we should perform more experiments. However, it is not said that my computer could bear more experiments as it is demanding for the processor.